Practical 10

Programs on Networking and Security

Networking and Security in C# refers to the use of C# programming to enable communication between computers, devices, applications, and services while protecting data and resources from unauthorized access. In networking, C# provides classes and libraries such as System.Net, System.Net.Sockets, TcpClient, TcpListener, HttpClient, and WebClient to communicate over networks using protocols such as TCP/IP and HTTP. For example, a C# application can connect to a web server, send an HTTP request, or communicate with another application through sockets. **Security in C#** focuses on protecting applications, users, and data through techniques such as authentication, authorization, encryption, secure communication, password protection, and input validation.

1. Get the Local IP Address

Objective: Create a C# program to display the IP address of the local computer.

```
using System;
using System.Net;

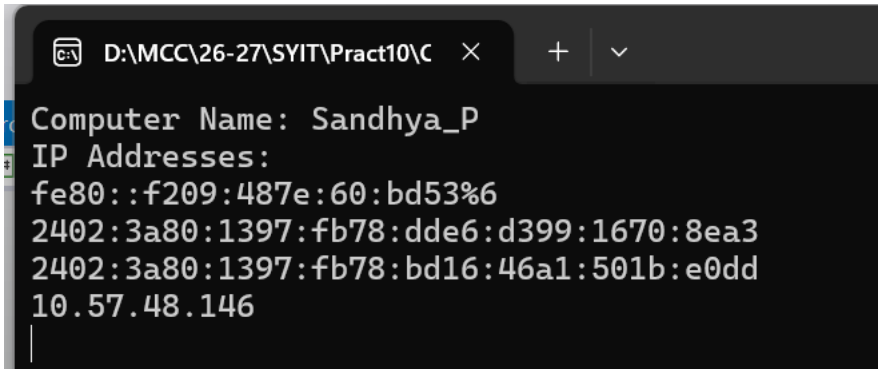
class Program
{
   static void Main()
   {
      string hostName = Dns.GetHostName();

      Console.WriteLine("Computer Name: " + hostName);

      IPAddress[] addresses = Dns.GetHostAddresses(hostName);

      Console.WriteLine("IP Addresses:");

      foreach (IPAddress address in addresses)
      {
         Console.WriteLine(address);
      }
      Console.ReadLine();
   }
}
```



Concepts: Dns, IPAddress, hostname, IP address.

2. Check Internet Connectivity

Objective: Create a C# program to check whether a website can be reached.

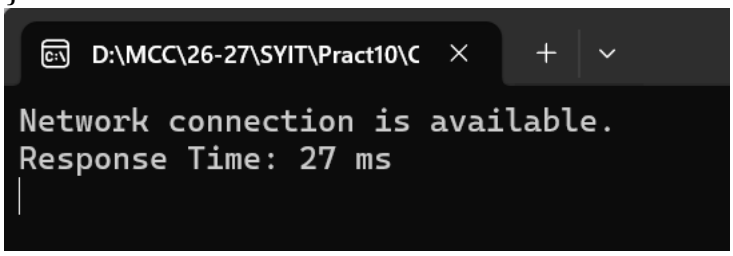
```
using System;
using System.Net.NetworkInformation;

class Program
{
   static void Main()
   {
      Ping ping = new Ping();

      try
      {
         PingReply reply = ping.Send("google.com");

         if (reply.Status == IPStatus.Success)
         {
            Console.WriteLine("Network connection is available.");
            Console.WriteLine("Response Time: " + reply.RoundtripTime + "
ms");
         }
         else
         {
            Console.WriteLine("Network connection is not available.");
         }
      }
      catch (Exception ex)
      {
         Console.WriteLine("Error: " + ex.Message);
      }
```

```
        Console.ReadLine();
    }
}
```



3. Convert Host Name to IP Address

Objective: Create a C# program to accept a website name and display its IP address.

```
using System;
using System.Net;

class Program
{
    static void Main()
    {
        Console.Write("Enter host name: ");
        string host = Console.ReadLine();

        try
        {
            IPAddress[] addresses = Dns.GetHostAddresses(host);

            Console.WriteLine("\nIP Addresses:");

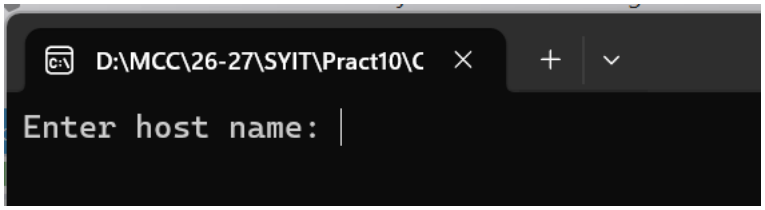
            foreach (IPAddress address in addresses)
            {
                Console.WriteLine(address);
            }
        }
        catch (Exception)
        {
            Console.WriteLine("Unable to find the host.");
        }

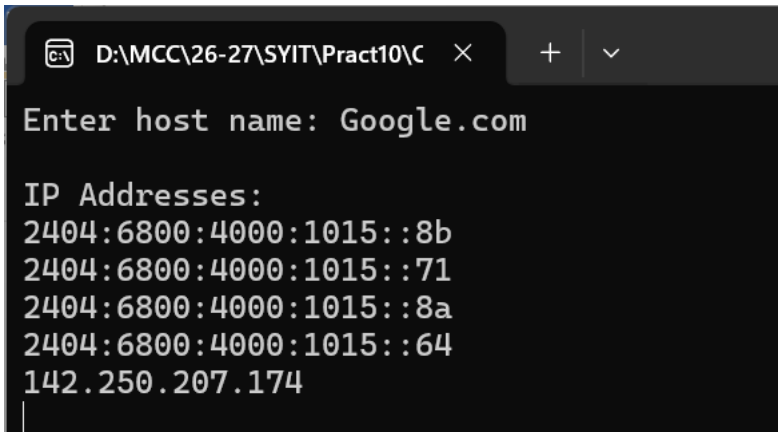
        Console.ReadLine();
```

```
    }
}
```

Sample input:

Enter host name: google.com





4. Get Host Name from an IP Address

Objective: Create a C# program to obtain the host name associated with an IP address.

```
using System;
using System.Net;

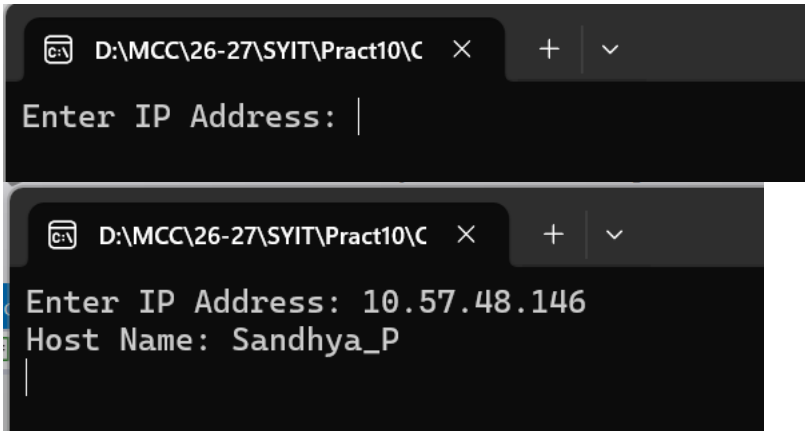
class Program
{
    static void Main()
    {
        Console.Write("Enter IP Address: ");
        string ip = Console.ReadLine();

        try
        {
            IPHostEntry host = Dns.GetHostEntry(ip);

            Console.WriteLine("Host Name: " + host.HostName);
```

```
        }
        catch (Exception)
        {
            Console.WriteLine("Host not found.");
        }

        Console.ReadLine();
    }
}
```



While Running program program will ask IP as input. For this Open CMD and Type **ipconfig** and press enter.

5. Password Strength Checker

Objective: Create a C# program to check whether a password meets basic strength requirements.

```
using System;

class Program
{
    static void Main()
    {
        Console.Write("Enter Password: ");
        string password = Console.ReadLine();

        bool hasUpper = false;
        bool hasLower = false;
        bool hasDigit = false;
```

```
        foreach (char c in password)
        {
            if (char.IsUpper(c))
                hasUpper = true;

            if (char.IsLower(c))
                hasLower = true;

            if (char.IsDigit(c))
                hasDigit = true;
        }

        if (password.Length >= 8 &&
            hasUpper &&
            hasLower &&
            hasDigit)
        {
            Console.WriteLine("Strong password.");
        }
        else
        {
            Console.WriteLine(
                "Password should contain at least 8 characters, " +
                "one uppercase letter, one lowercase letter, " +
                "and one digit.");
        }

        Console.ReadLine();
    }
}
```

